AI 활용 사례 — Unity 가상 조이스틱 구현 및 개선

1. AI 제작물 소개 : VirtualJoystick.cs

개인 Unity RPG 프로젝트에서 모바일 플레이어 조작을 위한 가상 조이스틱을 Codex를 활용해 구현했습니다.

터치 위치에 따른 조이스틱 이동, 드래그 방향 계산, 이동 범위 제한, 멀티터치 구분, 입력 해제 시 초기화 등의 기능을 포함합니다.

2. 실제 사용한 프롬프트 일부

① 기능 구현 및 분석 요청 — 9월 8일 15:40

"지금 Unity 프로젝트의 Ingame 씬에 조이스틱을 만들어 놓은 상태야. 조이스틱을 움직이면 안쪽 이미지도 입력 방향에 따라 움직이게 하고 싶어. 이동 범위는 바깥 원 크기 정도로 제한하고 싶은데, 씬 분석해봐."

② 기존 시스템과 연동 — 9월 8일 15:42

"근데 SetTarget()은 IngameSessionManager에서 호출하고 있거든? 기존 연결 방식은 유지하면서 VirtualJoystick 파일을 수정해 줘."

③ 요구사항 구체화 — 9월 8일 15:46

"안쪽 이미지 크기는 지금 설정한 그대로 유지하고 싶어. 이미지가 중앙에서 어느 방향으로 벗어났는지를 기준으로 이동 방향을 판단하면 좋겠어. 그리고 조이스틱을 얼마나 기울였는지에 따른 속도 가중치는 일단 없이, 방향만 적용하고 싶어."

④ 오류 확인 및 디버깅 — 9월 8일 15:51

"수정한 코드를 적용했는데 작동을 안 해. 캐릭터도 움직이지 않고 조이스틱 이미지도 반응이 없어. 혹시 포인터 입력 자체를 못 받고 있는 건 아닌지 확인해 봐."

⑤ 추가 기능 개선 — 9월 8일 16:17

"지금 VirtualJoystick에서 실제로 포인터 입력을 받는 영역이 어디야? 화면에 보이는 조이스틱 이미지보다 터치 영역을 좀 더 넓게 만들고 싶거든. 이미지 크기는 그대로 두고 입력 영역만 따로 확장해 봐."

※ 실제 AI와의 대화 내용을 바탕으로, 요청 의도는 유지하면서 일부 표현을 정리했습니다.

3. AI 활용 과정 및 의도

저는 AI를 개발 전반을 대신하는 도구가 아니라, '**개발 생산성을 높이기 위한 보조 도구**'로 활용하고 있습니다.

프로젝트의 전체 아키텍처와 공통 시스템, 모듈 간 인터페이스 및 상호작용은 직접 설계합니다. 이후 다른 시스템과의 의존성이 낮고 독립적으로 동작하는 하위 모듈의 구현에는 AI를 활용하여 개발 시간을 단축합니다.

또한 디버깅 과정에서 AI를 활용해 오류 원인을 분석하고 해결 방안을 검토합니다.

AI가 생성한 코드는 직접 읽고 동작 원리를 파악한 뒤, 기존 시스템과의 호환성 및 설계 의도를 기준으로 필요한 부분을 수정하고 통합합니다. 이후 실제 동작을 검증하고 추가적인 보완을 진행합니다.

즉, 개발자가 전체 구조와 방향을 결정하고, AI를 활용해 구현과 디버깅의 효율을 높인 뒤, 최종 검증과 통합을 직접 수행하는 방식입니다.



가상 조이스틱 개발에서는 먼저 AI에 필요한 기능의 구현을 요청한 뒤, 생성된 코드를 직접 읽고 기존 플레이어 입력 시스템과의 연동 방식을 검토했습니다.

이후 조이스틱의 이미지 크기를 유지하면서 중심점을 기준으로 이동 방향을 계산하고, 드래그 거리에 따른 속도 가중치를 제거하도록 요구사항을 구체화했습니다.

실제 테스트에서 조이스틱 이미지와 캐릭터가 움직이지 않는 문제가 발생했을 때는 AI와 함께 원인을 분석하고 수정했습니다. 또한 조작 편의성을 높이기 위해 시각적인 조이스틱 영역과 실제

터치 입력 영역을 분리하는 방식도 검토했습니다.

이처럼 AI가 생성한 코드를 그대로 사용하는 것이 아니라, **코드 분석 → 수정 → 테스트 및 디버깅 → 추가 보완** 과정을 거쳐 프로젝트에 적용했습니다.

이를 통해 독립적인 기능의 구현에 드는 시간을 줄이고, 전체 시스템 설계와 핵심 게임 로직 개발에 더 집중할 수 있었습니다.